Bloc 3 Développement de la solution digitale M Motors

Adam GHAJDAOUI

Bachelor HETIC - C3.1, C3.2 et C3.3 - 5 novembre 2026

Accès à la réalisation

Description	Valeur
Lien Git frontend	https://github.com/Adam-84/m-motors-bloc3/tree/main/frontend
Lien Git backend	https://github.com/Adam-84/m-motors-bloc3/tree/main/backend
Lien de l'application	https://m-motors-bloc3.onrender.com/
Login administrateur	admin@mmotors.example
Mot de passe administrateur	Admin@20262027
Login client	client@mmotors.example
Mot de passe client	User@20262027

Cette réalisation reprend le projet M-Motors étudié dans mes blocs 1 et 2 : permettre l'achat ou la location avec option d'achat d'un véhicule d'occasion, puis traiter le dossier en ligne. Le front et l'API sont déployés ensemble sur Render (région Frankfurt) avec une base PostgreSQL. Le dépôt est un monorepo public : les deux liens ci-dessus pointent vers ses sous-dossiers frontend et backend.

L'hébergement gratuit met l'application en veille après 15 minutes d'inactivité : le premier chargement peut prendre une minute. L'état du service est visible sur https://m-motors-bloc3.onrender.com/actuator/health.

Les comptes, véhicules, documents de test et tarifs sont fictifs. Ces identifiants ne donnent accès à aucun compte personnel.

1 Continuité avec les blocs 1 et 2

Les choix que je conserve

Le bloc 1 séparait le parcours client et le back-office. Je garde cette séparation dans l'interface, tout en utilisant une API commune pour les règles métier. Le client consulte le catalogue, ouvre un dossier et transmet ses pièces. L'administrateur gère les offres et prend une décision sur les dossiers soumis.

Je conserve React et les échanges HTTP REST proposés dans le bloc 1. Pour ce bloc, je choisis Java 21 et Spring Boot côté serveur. Le bloc 1 citait Node.js dans son estimation de budget ; ce choix d'implémentation évolue donc, sans modifier le découpage front / API / base relationnelle. L'énoncé propose Java mais précise qu'aucune contrainte technique n'est imposée.

Le bloc 2 retenait Scrum, un backlog Trello et Cypress. Ces choix restent la référence méthodologique. Le board transmis dans le bloc 2 est https://trello.com/b/Lc3yb7Cp/m-motors-examen-bloc-2. Son contenu et son accessibilité publique devront être revérifiés au moment du dépôt ; ce dossier ne prétend pas l'avoir mis à jour.

Les ajustements nécessaires

Dans le MVP du bloc 1, le suivi client et la bascule achat/location pouvaient être reportés. Le bloc 3 demande explicitement de les réaliser : ils sont donc inclus dans cette application. Le compte administrateur unique décrit comme simplification en B1 ne dispense pas du contrôle des droits : l'inscription publique crée toujours un CLIENT.

La réalisation est un démonstrateur fonctionnel. Elle n'est pas dimensionnée pour les 800 salariés et le million de clients du contexte de l'énoncé. Le load balancer, la réplication et le stockage objet du schéma cible B1 restent des évolutions d'exploitation, pas des composants déjà déployés ici.

Périmètre métier

Inclus dans cette version	À contractualiser ou traiter ensuite
Recherche achat/location, fiche et budget	Calcul financier complet de la LOA
Compte client et dossier dématérialisé	Signature électronique et paiement
Options de services et total mensuel	Apport, durée et valeur de rachat
Suivi et décision administrateur	Notifications automatiques par e-mail
Catalogue administrateur et bascule	Connexion réelle au partenaire financier

Les prix de services sont des hypothèses explicites de démonstration : assurance 35 €/mois, assistance 8 €/mois, entretien 25 €/mois et contrôle technique 5 €/mois. Je ne les présente pas comme des tarifs transmis par M-Motors.

2 Architecture de la réalisation

Une application organisée en composants

Le navigateur charge les fichiers React et appelle les routes /api du même serveur. Spring Security vérifie la session et le jeton CSRF avant l'appel du contrôleur. Le contrôleur valide les formats d'entrée ; MotorsService applique les règles métier dans une transaction puis interroge la base avec JDBC. Flyway crée et versionne le schéma.

```
Navigateur React
       | HTTP local / HTTPS sur Render
Spring Boot et Spring Security
       | API Controller : formats et routes
MotorsService et BusinessRules
       | requêtes paramétrées et transactions
Base relationnelle : H2 local / PostgreSQL cible
       | utilisateurs, véhicules, dossiers, options, pièces
```

Les classes ont des responsabilités distinctes : SecurityConfig configure l'authentification, Models décrit les entrées validées, ApiController expose les routes, MotorsService porte les traitements transactionnels et BusinessRules regroupe les règles qui se testent sans serveur. ApiErrors centralise les réponses d'erreur.

Modèle de données

Un utilisateur possède plusieurs dossiers ; chaque dossier concerne un véhicule. Les services sélectionnés sont associés au dossier dans dossier_option. Une pièce appartient à un dossier et à une catégorie, avec une contrainte d'unicité par couple dossier/catégorie. Les clés étrangères empêchent les pièces orphelines.

Le prix d'achat, la mensualité et le type d'offre sont copiés dans le dossier à sa création. Ce sont des valeurs contractuelles de la demande, pas une duplication accidentelle du catalogue : une modification ultérieure de prix ne doit pas réécrire une demande déjà ouverte.

Choix de stockage et de livraison

En local, H2 stocke les données sur disque dans backend/data. En cloud, le profil cloud utilise PostgreSQL avec chiffrement de la connexion. Le schéma initial est contenu dans V1__initial_schema.sql. L'historique de l'ancien SI n'est pas fourni : aucune migration des données réelles n'est revendiquée.

Pour ce périmètre, les pièces sont conservées en binaire dans la base. Une sauvegarde de la base contient ainsi aussi les documents. À l'échelle entreprise, je reprendrais le stockage objet privé proposé en B1, avec analyse antivirus, politique de conservation et contrôle d'accès signé. Le déploiement préparé utilise un conteneur unique, sans cluster Kubernetes.

3 Préparer Git et organiser les branches

Organisation du dépôt

Je prépare un monorepo : frontend contient React et Cypress ; backend contient l'API Java, les migrations SQL et les tests JUnit ; docs regroupe la conception, la procédure de déploiement et ce dossier. La séparation permet de retrouver immédiatement les deux composants tout en versionnant ensemble une évolution qui modifie leur contrat API.

Le dossier final affiche deux liens explicites vers les sous-dossiers frontend et backend du dépôt public https://github.com/Adam-84/m-motors-bloc3. Un monorepo n'empêche pas cette lecture.

Principe des branches

J'applique un trunk-based simplifié avec des branches courtes. `main` est toujours stable et déployable ; toute évolution passe par une pull request.

Branche	Rôle	Règles
main	Version stable, déployée	Aucun commit direct ; pull requests avec CI verte uniquement
feat/US-xx-nom	Une user story	Part de main, courte durée, supprimée après fusion
fix/BUG-xx-nom	Correction d'un bug	Part de main, avec un test de non-régression
docs/nom	Documentation seule	Même circuit de pull request

Cycle cible pour chaque story : branche depuis main, test écrit en premier, implémentation, pull request avec modèle de description, fusion après CI verte et relecture du diff, suppression de la branche, déploiement depuis main, tag de version. La règle est écrite dans CONTRIBUTING.md et le modèle de PR dans .github/pull_request_template.md.

Écart reconnu entre ce cycle et l'historique. Le premier incrément, le parcours complet US-01 à US-10, a été livré en un seul commit sur feat/mvp (6696236), sans branche ni pull request par story. L'historique ne permet donc pas de vérifier une branche par story ni des tests écrits avant le code pour ces dix stories. Je ne le reconstruis pas après coup. Le cycle ci-dessus s'applique depuis la correction du dépôt : une branche courte par amélioration (refactor/, fix/, test/, docs/), un commit par intention, une pull request et la CI verte avant fusion. Les branches et pull requests visibles sur GitHub sont la seule preuve revendiquée.

Protection de main (réglage GitHub) : pull request obligatoire et contrôles de statut requis avant fusion. Ce réglage n'est pas activé sur ce dépôt. Depuis la correction, les évolutions passent par une pull request, par discipline et non par blocage technique.

Contenu et qualité des commits

Les messages suivent les Conventional Commits (feat, fix, refactor, test, docs). Le commit initial du MVP regroupe pourtant tout le parcours : c'est le défaut que je corrige en ne livrant plus qu'un changement par commit. La CI construit React, exécute les tests et conserve le rapport de couverture. Les mots de passe réels, bases locales et fichiers de travail sont exclus par .gitignore ; les comptes de démo n'ont aucune donnée réelle.

Exemple d'itération cible : créer feat/US-03-dossier, écrire un test du comportement attendu, implémenter la route et l'écran, vérifier les tests, ouvrir une pull request puis fusionner après revue. Une story n'est DONE que si ses critères fonctionnels et la DoD sont respectés.

4 Développer une user story du backlog

Démarche appliquée

- Prendre une story priorisée avec le PO, puis relire son besoin et sa valeur utilisateur.
- Vérifier la DoR : rôle concerné, règles métier, entrées, résultats attendus et critères d'acceptation suffisamment précis.
- Découper la story en données, API, interface et tests, sans perdre le parcours utilisateur complet.
- Créer une branche liée à son identifiant et écrire les tests de ses règles les plus risquées.
- Exécuter les tests avant l'implémentation pour vérifier qu'ils détectent réellement le comportement manquant.
- Développer la règle métier, l'API et l'interface en respectant le contrat convenu.
- Exécuter les tests unitaires, les tests d'intégration et le parcours navigateur concerné ; corriger puis vérifier les non-régressions.
- Faire relire le changement et mettre à jour la documentation utile.
- Construire l'artefact, déployer sur l'environnement retenu, vérifier le parcours avec le PO et conserver une preuve du commit livré.
- Mettre la story DONE uniquement lorsque les critères d'acceptation et la DoD sont satisfaits.

Exemple de story

US-04 : « En tant que client connecté, je veux transmettre les pièces de mon dossier afin que le service commercial puisse étudier ma demande sans échange papier. »

Les critères sont concrets : accepter un PDF/JPEG/PNG lisible de 5 Mo maximum ; rattacher la pièce au bon dossier ; autoriser seulement le propriétaire ou un administrateur ; permettre le remplacement en brouillon ; refuser la soumission si identité ou domicile manque ; bloquer la modification après soumission.

Le test doit couvrir la réussite, mais aussi un autre client qui devine l'identifiant du dossier, un fichier exécutable renommé .pdf et un document tronqué ayant une signature valide. Les messages visibles doivent permettre au client de comprendre et corriger sa demande.

DoR et DoD

La DoR rend la story prête à entrer dans le développement. Elle ne signifie pas que tout élément du backlog doit déjà être prêt : les besoins lointains peuvent être affinés plus tard.

La DoD reprend celle du B2 et la rend vérifiable : critères d'acceptation validés, tests verts, absence de bug bloquant, changement relu, documentation à jour et incrément déployable. La vérification du déploiement fait partie de la clôture : l'application est déployée sur https://m-motors-bloc3.onrender.com/ depuis la branche main, et la santé du service est vérifiée sur /actuator/health (statut UP).

5 Périmètre des user stories réalisées

Story	Valeur et critère d'acceptation principal	Trace dans le projet
US-01 Catalogue	Le visiteur filtre achat/location et budget, puis consulte le modèle, les kilomètres et le prix adapté à l'offre	GET /api/vehicles et interface catalogue
US-02 Compte	Le visiteur crée un compte CLIENT et se connecte ; une tentative de s'attribuer ADMIN est ignorée	register, login, me, SecurityConfig
US-03 Demande	Le client ouvre un dossier pour un véhicule disponible ; l'offre et les prix sont figés	createDossier et contraintes SQL
US-04 Pièces	Le client dépose identité et domicile ; seules ses pièces sont accessibles dans son espace	upload, document, PDFBox et ImageIO
US-05 Soumission	Le dossier complet passe de BROUILLON à SOUMIS et devient non modifiable côté client	submit et BusinessRules.requireComplete
US-06 Suivi	Le client retrouve ses dossiers, leur statut et le motif de décision	GET /api/dossiers et écran de suivi
US-07 Catalogue admin	L'administrateur ajoute une offre ; un client ne peut pas appeler ces routes	/api/admin/vehicles et validation des entrées
US-08 Bascule	L'administrateur change achat/location seulement sans dossier actif	switchOffer et verrouillage véhicule
US-09 Instruction	L'administrateur valide ou refuse un dossier soumis ; le refus exige un motif	decide et validation transactionnelle
US-10 Options LOA	Le client choisit des services autorisés ; aucun service LOA ne peut être ajouté à un achat	BusinessRules.validateOptions et monthlyTotal

Ces identifiants servent de correspondance locale avec les fonctionnalités du B1. Le contenu exact des cartes du board B2 n'a pas été importé automatiquement : il faut aligner les identifiants Trello avant de clôturer les cartes.

Règles d'intégrité

La validation commerciale rend le véhicule indisponible avec une mise à jour atomique : UPDATE vehicle SET available=FALSE WHERE id=? AND available=TRUE. Si aucune ligne n'est modifiée, un autre dossier a déjà obtenu le véhicule et la décision est refusée avec un conflit HTTP 409.

Plusieurs prospects peuvent déposer un dossier, mais l'ouverture d'un brouillon ne réserve pas le véhicule. Ce comportement est indiqué dans l'interface. Le verrouillage transactionnel évite qu'une soumission et un remplacement de pièce se croisent sans contrôle du statut.

6 Mobiliser les outils adaptés aux objectifs

React et API Java

React reprend le choix du B1. Les écrans partagent l'état de connexion et les éléments d'interface ; les listes sont rendues à partir de la réponse réelle de l'API. Les champs utilisent des labels et les messages d'erreur sont annoncés avec un rôle d'alerte. Les feuilles CSS adaptent le catalogue et les formulaires aux petits écrans.

Spring Boot fournit le serveur HTTP et l'assemblage des composants. Spring Security prend en charge la session, la déconnexion, le contrôle de rôle et CSRF. Réutiliser ce cadre limite la quantité de code de sécurité spécifique à écrire et à maintenir.

Bean Validation impose les formats et bornes des entrées : adresse e-mail, longueurs, prix positifs et nombre de décimales. Le rôle demandé par un navigateur n'est jamais utilisé pour déterminer les droits de l'inscription. Les règles métier sont aussi vérifiées côté serveur ; les contraintes HTML ne sont qu'une aide à la saisie.

Base relationnelle et transactions

JDBC rend visibles les requêtes paramétrées et les jointures. Flyway conserve la version du schéma. Les contraintes de base protègent les relations même si une erreur applicative survient. Les montants utilisent BigDecimal côté Java et NUMERIC côté SQL, afin d'éviter les approximations des nombres à virgule flottante.

Tests et construction

JUnit 5 et AssertJ vérifient les règles et les contrats HTTP. MockMvc traverse les filtres de sécurité, le contrôleur et la base H2 de test. Il ne remplace pas la base par un mock. JaCoCo mesure les lignes, instructions et branches exécutées pendant la suite.

Cypress est conservé conformément au B2 pour vérifier les parcours dans un navigateur. Maven construit le backend ; Vite construit le front. Le Dockerfile prépare un artefact reproductible et exécute les tests pendant sa construction.

Appui sur les cours

Le cours Python « Tests unitaires, TDD et Gherkin » apporte la séparation des niveaux de tests et le cycle échec / réussite / amélioration. Je transpose ces principes à JUnit, sans prétendre utiliser unittest dans une application Java. Les cours SQL sur les jointures et les utilisateurs guident les clés étrangères, l'identification des propriétaires et les rôles. Le cours sur les exceptions aide à distinguer erreur de saisie, ressource absente et incident inattendu.

7 Tests unitaires et couverture

Résultats mesurés

Les valeurs ci-dessous sont calculées à partir des fichiers produits par Maven et JaCoCo lors de la génération de ce dossier. La commande de référence est mvn clean verify dans backend. La compilation React précède cette commande pour intégrer l'interface au JAR.

Mesure	Résultat local
Tests JUnit exécutés	32
Échecs JUnit	0
Erreurs JUnit	0
Couverture des lignes backend	91,46 % soit 182 / 199 lignes
Couverture des branches backend	73,98 %
Couverture des instructions backend	90,14 %
Parcours Cypress	2 tests de bout en bout, 0 échec

Le pourcentage porte sur les classes de production Java instrumentées par JaCoCo, y compris configuration, contrôleurs et gestion d'erreurs. Ce n'est pas une mesure de couverture du code React ni une preuve que tous les défauts possibles sont éliminés. Les classes et les compteurs détaillés restent disponibles dans target/site/jacoco/index.html.

Ce qui est vérifié

Les tests unitaires couvrent le total des services sélectionnés, les options interdites à l'achat, les services inconnus, les deux pièces obligatoires, les transitions de statut, le refus motivé et la lecture des formats de fichiers. Les fixtures positives sont de vrais PDF et images générés par les outils de test.

Les tests d'intégration vérifient notamment l'inscription sans escalade de privilège, le hachage du mot de passe, la connexion réelle, la déconnexion, CSRF, la recherche, le dépôt privé des pièces et l'instruction d'un dossier. Un test vérifie que 201 dossiers restent visibles dans l'historique administrateur.

Portée de la validation

H2 facilite l'exécution locale. La CI prévoit une seconde exécution sur PostgreSQL 17 afin de détecter les différences SQL ; cette exécution distante ne doit pas être considérée comme réussie avant d'avoir réellement tourné. Aucun test de charge représentatif du million de clients, audit d'intrusion indépendant ou exercice cloud de reprise n'a encore été réalisé.

8 Justifier les mesures de sécurité

Risque	Mesure implémentée	Vérification ou limite
Attribution du rôle admin à l'inscription	Le serveur impose CLIENT et ignore les champs de rôle supplémentaires	Test d'inscription avec role=ADMIN
Vol d'un mot de passe stocké	Hachage BCrypt avec facteur 12 ; aucun mot de passe clair en BDD	Test du hash et de connexion
Action forgée depuis un autre site	Jeton CSRF obtenu par l'interface et vérifié sur les mutations	POST sans CSRF refusé
Lecture du dossier d'un autre client	Filtre par propriétaire et contrôle d'appartenance sur lecture et pièces	Autre client : réponse 404
Appel du back-office par un client	Routes /api/admin réservées à ROLE_ADMIN	Réponse 403 pour CLIENT
Injection SQL	Paramètres JDBC liés et valeurs d'offre validées	Pas de concaténation des valeurs utilisateur
Fichier trompeur ou trop lourd	5 Mo, signature et parsing PDF/ImageIO ; limites de pages et pixels	Exécutable et fichier tronqué refusés
Contenu actif dans un justificatif	Stockage privé, téléchargement en pièce jointe, nosniff et CSP	Aucun antivirus intégré à cette version
Vol de session via script	Cookie HttpOnly et SameSite=Lax ; session expirant après 30 min	Secure activé dans le profil cloud
Fuite dans les journaux	Identifiants techniques et événements, sans contenu des pièces ou mots de passe	Collecte et rétention cloud à configurer

Les mots de passe acceptés ont au moins 12 caractères et au plus 64 caractères, avec une limite supplémentaire de 72 octets UTF-8 liée à BCrypt. Les requêtes de validation et de décision passent par des entrées typées et des contrôles métier. Les messages inattendus ne renvoient pas de stack trace au navigateur.

Les justificatifs ne sont pas accessibles comme fichiers statiques. La route de téléchargement vérifie le dossier associé avant de retourner le contenu, puis ajoute Cache-Control: no-store et Content-Disposition: attachment. Un identifiant difficile à deviner ne suffit pas : le contrôle de propriété reste obligatoire.

Pour une utilisation avec de vraies données, les prochaines mesures sont l'analyse antivirus, la limitation de débit, la conservation limitée et la supervision de sécurité. La démo ne revendique pas une conformité ou un audit complet.

9 Identifier et corriger les erreurs

Démarche d'investigation

Je pars d'un comportement reproductible, relève le résultat attendu et le résultat obtenu, puis cherche la couche responsable : interface, contrat HTTP, règle métier ou base. Je réduis ensuite le scénario jusqu'au plus petit test capable de reproduire le problème. Le correctif est suivi de ce test et de la suite complète.

La revue de code a été utilisée pour compléter les tests, notamment sur les cas que les premières fixtures ne représentaient pas correctement. Les exemples suivants correspondent à des défauts réellement constatés pendant la réalisation.

Problème constaté	Cause identifiée	Correction et preuve
La création du schéma échoue sur H2	year est interprété comme un mot réservé dans ce contexte SQL	Colonne renommée model_year ; migration Flyway et suite d'intégration exécutées
Un pseudo-PDF de cinq octets est accepté	Vérifier seulement %PDF- ne prouve pas que le document est lisible	Parsing PDFBox et décodage ImageIO ; test rejetant les signatures tronquées
L'ancien historique disparaît après 200 dossiers	LIMIT 200 sans navigation vers la suite	Retrait de la limite des listes de dossiers pour ce périmètre ; test avec 201 dossiers

Exemple de cycle de correction

Le bug de code retenu est celui du contrôle des pièces justificatives, dans BusinessRules.fileType.

1. **Symptôme** : un fichier de cinq octets commençant par %PDF- est accepté comme pièce d'identité. 2. **Résultat attendu / obtenu** : refus avec le message « Document illisible » ; l'ancienne version répondait par un succès. 3. **Couche responsable** : la règle métier. Le contrôleur et la base ne font que transmettre les octets. 4. **Plus petit test** : un tableau d'octets tronqué passé à fileType, sans HTTP. C'est le test rejectsTruncatedPdfAndImagesEvenWithCorrectSignature de BusinessRulesTest, qui couvre aussi les en-têtes JPEG et PNG tronqués. 5. **Cause** : la vérification s'arrêtait à la signature, qui prouve le début du fichier et non sa lisibilité. 6. **Correction** : lecture réelle du PDF avec PDFBox (non chiffré, 1 à 100 pages) et décodage des images avec ImageIO (20 mégapixels maximum). 7. **Preuve** : ce test et la suite complète (32 tests) passent, puis la CI.

L'historique Git ne montre pas l'ordre test puis correctif pour ce bug, corrigé pendant la réalisation initiale. Le test de non-régression le protège désormais. Pour tout nouveau bug, l'ordre sera vérifiable : une branche fix/BUG-xx, un premier commit contenant le test qui échoue, un second contenant la correction, puis une pull request.

Cette correction améliore la validité structurelle des pièces, mais elle ne remplace ni l'analyse antivirus ni l'évaluation humaine de leur contenu. Un PDF valide peut contenir une pièce qui n'est pas le justificatif demandé : l'administrateur conserve la décision métier.

Optimisations proportionnées

Des index ciblent les dossiers par propriétaire et véhicule, ainsi que les véhicules par offre et disponibilité. Les listes résument les dossiers sans charger le binaire des pièces. Les fichiers sont chargés uniquement lors du téléchargement. La taille et la complexité des entrées sont bornées.

La suppression du LIMIT est un compromis pédagogique explicite. Une version entreprise devra paginer et filtrer les résultats, tout en conservant l'accès aux anciens dossiers ; charger tous les dossiers d'un million de clients n'est pas l'architecture cible.

10 Surveiller les erreurs et l'application

Journaux utiles à l'exploitation

Les opérations importantes produisent un événement applicatif : dossier_created, document_uploaded, dossier_submitted, dossier_decided, vehicle_created et offer_changed. Le journal indique l'identifiant technique concerné et, si utile, le statut ou la catégorie de pièce.

```
event=dossier_submitted dossier=<identifiant technique>
event=dossier_decided dossier=<identifiant> status=VALIDE
```

Ces lignes décrivent le format implémenté ; elles ne sont pas des preuves d'un service cloud déjà exploité. Le contenu des justificatifs, les mots de passe et les données de session ne sont pas journalisés par ces événements.

Pour une exception inattendue, ApiErrors génère une référence, consigne le type d'incident et renvoie une réponse générique contenant cette même référence. Cela permet de rechercher un incident sans révéler le détail technique au client. La journalisation actuelle reste volontairement minimale ; elle ne contient pas encore un audit complet avec identité de l'opérateur et rétention opposable.

Codes de réponse et état de santé

Situation	Réponse attendue
Saisie ou format invalide	400 avec message compréhensible
Session absente ou identifiants incorrects	401
Rôle insuffisant ou CSRF invalide	403
Ressource absente ou non possédée	404
Conflit d'état métier	409
Fichier trop volumineux	413
Incident non prévu	500 avec référence de corrélation

L'endpoint /actuator/health vérifie l'état de l'application et la connexion à la base, avec des détails masqués au public. Le plan d'exploitation prévoit une alerte sur indisponibilité, sur hausse du taux de 5xx, sur latence et sur échec des sauvegardes. Les seuils devront être établis après mesure ; aucune alerte distante n'est déclarée active dans cette version locale.

L'interface affiche une erreur en cas d'échec d'appel et conserve les réponses métier utiles. Les tests de navigateur complètent cette surveillance pendant la validation ; ils ne remplacent pas un suivi continu de la production.

11 Livraison et déploiement

Validation locale d'abord

Le script start-local.ps1 compile React, lance les tests Maven et démarre le JAR. La variante -SkipBuild utilise le JAR déjà construit. Le catalogue, le compte client et l'administration sont servis sur le même port. Les données H2 restent dans backend/data entre deux redémarrages.

Les parcours à vérifier sont : filtrer les offres ; ouvrir une fiche ; s'inscrire ; sélectionner les services d'une location ; ouvrir un dossier ; transmettre les deux pièces ; soumettre ; consulter le suivi ; se connecter comme administrateur ; ouvrir le dossier et enregistrer une décision motivée.

Publication sur Render

Le Dockerfile effectue la construction React et les tests Java avant de produire une image d'exécution. Le workflow GitHub Actions fourni prépare également les tests sur PostgreSQL et Cypress. Le fichier render.yaml (Blueprint) décrit le service web Docker et la base PostgreSQL managée, région Frankfurt, plan gratuit. Il a été appliqué depuis la branche main : l'application est en ligne en HTTPS avec les comptes de démonstration du tableau d'accès.

J'ai retenu Render après l'essai local. Son offre gratuite met les applications en veille et expire la base au bout de 30 jours [S9] : il faut tenir compte du calendrier de correction.

Le déploiement dès la première story DONE est une bonne pratique recommandée dans l'énoncé. Dans cette réalisation, le développement a d'abord été validé en local ; la première mise en ligne regroupe donc l'ensemble des stories (US-01 à US-10) et non la seule première. Je ne présente pas de preuve historique artificielle : la preuve est le déploiement de main, visible sur Render et sur l'URL publique. Pour les itérations suivantes, chaque story DONE sera déployée dès qu'elle est fusionnée.

Reprise et migration

Les valeurs de RPO et RTO doivent être convenues avec le PO. À titre de proposition pour la cible entreprise, je retiens RPO de 15 minutes et RTO d'une heure. Ces valeurs ne sont pas des performances mesurées de la démo.

Pour les atteindre, la cible devra disposer de sauvegardes et d'une récupération temporelle compatibles, d'un stockage protégé et d'un exercice chronométré de restauration. Les sauvegardes devront inclure les pièces. Une sauvegarde quotidienne seule ne suffit pas à garantir 15 minutes de perte maximale.

La migration réelle de l'ancien SI reste à construire à partir d'un export connu : mapping, nettoyage, chargement par lots, contrôle des volumes et relations, répétition sur préproduction, sauvegarde puis bascule avec retour arrière. Flyway gère ici les versions du schéma ; il ne prouve pas à lui seul la migration d'un million de clients.

12 Sources et vérifications avant le dépôt final

Documents de continuité

[S1] Énoncé du Bloc 3 M-Motors, fichier ff506b2c954457f3075a742ba7720bf96d5e5a20.pdf, 8 pages. Les questions C3.1 figurent page 5 ; les livrables et obligations figurent pages 7 et 8.

[S2] Copie Bloc 1 d'Adam GHAJDAOUI : fonctionnalités, MVP, choix React/API/cloud, architecture et pistes de migration. Les coûts estimés du B1 ne sont pas présentés ici comme des dépenses réellement engagées.

[S3] Copie Bloc 2 d'Adam GHAJDAOUI : choix Scrum, DoR/DoD, backlog Trello et Cypress. Les décisions méthodologiques ont servi de continuité ; leur réalisation passée n'est pas inventée.

Cours utilisés

Racine des notes : G:/Mon Drive/ObsidianVault/Notes/Cours/BLOC 3.

[S4] Python / « 10. Python - Tests unitaires, TDD et Gherkin » : organisation des tests, assertions, cycle Red-Green-Refactor et critères d'acceptation.

[S5] Python / « 8. Python - Améliorer la qualité du code (Exceptions + PEP 8) » : saisies, ressources externes et règles métier comme sources d'erreurs ; gestion spécifique avant gestion générique. Les conventions PEP 8 ne sont pas imposées au code Java.

[S6] Créer et administrer une base de données relationnelle / « 2. Modèle conceptuel de données et modèle logique de données » et « 12. Jointures SQL » : associations, clés, séparation des données et jointures.

[S7] Même dossier / « 13. Gestion des utilisateurs » : identifiant, login, mot de passe et rôle. Ces concepts sont appliqués avec Spring Security et BCrypt.

Références techniques

[S8] Spring Boot 3.5, exigences système : https://docs.spring.io/spring-boot/3.5/system-requirements.html. PDFBox : https://pdfbox.apache.org. Les versions exactes sont fixées dans pom.xml et package-lock.json.

[S9] Render, offre gratuite et spécification Blueprint : https://render.com/docs/free et https://render.com/docs/blueprint-spec, consultées le 5 novembre 2026.

Retour sur le premier dépôt

Le premier dépôt a reçu trois critiques. Voici l'état de chacune, sans plus que ce que le dépôt prouve.

Critique	Constat	Réponse
Historique Git : un seul commit pour toute l'application, pas de branche par story	Exact : le parcours US-01 à US-10 est dans le commit 6696236 de feat/mvp	Écart reconnu en partie 3. Chaque amélioration passe désormais par sa branche, ses commits et sa pull request
Code compact et peu lisible	Exact : lignes denses, plusieurs instructions par ligne, résultats en Map	BusinessRules réécrit en méthodes courtes à constantes nommées, branche refactor/lisibilite-regles-metier, 32 tests inchangés. MotorsService reste à découper en services par domaine
Démarche C3.3 centrée sur la configuration	Exact : l'exemple de la première version était une erreur de déploiement	Démarche reprise sur un bug de code, le faux PDF accepté, en partie 9

Ce tableau décrit l'état au moment de la génération du PDF. Les branches, pull requests et la CI verte se vérifient sur https://github.com/Adam-84/m-motors-bloc3.

État de la livraison et limites assumées

Les sources sont publiées dans un dépôt public avec ses liens frontend et backend en première page, l'application est déployée sur Render et les comptes d'évaluation distants sont renseignés et testés. Les parcours ont été vérifiés depuis l'URL publique.

Restent à renforcer dans les prochaines itérations : un audit de sécurité indépendant, un test de charge représentatif, un suivi continu de la production et la migration réelle de l'ancien SI. Je suis ouvert aux critiques de l'évaluateur sur ces points et je les traiterai par des branches fix/ et feat/ avec pull request, tests et redéploiement.